

EBIN BABU THOMAS

AI Engineer — LLM agents, evaluation engineering & AI control

ebinbt.dev | ebinbabuthomas@gmail.com | github.com/ebt55 | Mavelikara, Kerala, India (Remote)

SUMMARY

Engineer-researcher working on AI control and evaluation engineering: agent monitoring, model diffing, red-teaming, and preregistered benchmarks whose answer keys are sealed before the first run. Before that, 3.5 years shipping LLM, RAG and agent backends end-to-end for startup clients in the US, Canada, Europe and Australia. Open-source contributor with merged upstream work. Every number below comes from a run that reproduces from the linked repository, and the failed experiments are published beside the ones that worked.

TECHNICAL SKILLS

Core: Python, FastAPI, LLM Agents & Tool Use (Claude Agent SDK, Gemini API, PydanticAI, LangGraph), MCP (FastMCP), RAG, Prompt & Context Engineering, Playwright, pytest, Docker, Git

Proficient: PostgreSQL / SQL, MongoDB, Vector DBs (Qdrant), PyTorch, Hugging Face Transformers / PEFT, Fine-tuning (LoRA / QLoRA, DPO), LLM Serving (vLLM, Modal), NLP (sentence-transformers, Whisper), MLOps (GitHub Actions CI/CD, OpenTelemetry), Pandas, AWS, GCP, Kubernetes (GKE), OAuth2 / JWT, Terraform, Linux, uv

Evaluation engineering & measurement: eval harness design, preregistered experiment design, red-teaming (reward-hacking environments), model diffing, blind grading, LLM-as-judge agreement checks, bootstrap & permutation statistics, log-probability metrics, crash / kill-point testing, reproducible results (SHA-256-pinned artifacts), OpenTelemetry, Langfuse

Familiar: LangChain, Data Streaming (Bytewax), Next.js, TypeScript

INDEPENDENT PROJECTS & RESEARCH

diffing-agent-bench — Sealed Benchmark for Black-Box Model-Diffing Agents

Aug – Sep 2026

Work sample for the MATS 12.0 application, Neel Nanda stream, Sep 2026 — github.com/ebt55/diffing-agent-bench (Python, Qwen3.5-9B LoRA finetunes, vLLM, preregistered blind grading)

- Built a sealed answer key for model-diffing agents: **five LoRA finetunes of Qwen3.5-9B** that differ only in what was planted in their training answers, with the label map committed before any run. The published recipe and four cheaper conditions audited them blind, in two phases against a rubric frozen in advance, under a preregistration carrying 10 committed amendments. Built solo (LLM coding agents); every number regenerates from committed scripts.
- The agent never asked what it needed to ask: **0 of 13 attempts** on the PostgreSQL rung put a database question to either model. A fixed 50-prompt battery found the same planted bias at **\$0.15 per detection against \$3.14 for the agent**.
- The auditor is itself a failure mode: **8 of 40 frontier audits ended with no verdict** because a provider classifier stopped them, and 6 of 20 runs against literally identical weights hit the same cutoff. Every detection cell here is 5 runs or fewer, so one run flips any of them.

IncidentGate — Test Lab for a Safety-Gated Incident-Response Agent

Aug – Sep 2026

github.com/ebt55/incidentgate (Python 3.12, LangGraph, MCP (FastMCP), PostgreSQL, OpenTelemetry → Langfuse, Docker Compose, uv, GitHub Actions)

- A measurement lab, not a product: it tests how three safeguards — fixed policy rules, an AI second-opinion monitor, and human sign-off — change what an incident-fixing agent does when it crashes, is fed misleading evidence, or receives hostile input. **No change goes through without every gate plus a one-time approval token issued by a person.**
- Crash-safety, proven: the agent process was force-killed at every step of every scenario — **594 kill points across 27 scenarios — and all 434 measurable cases recovered to exactly the expected end state:** zero incidents lost, zero actions run twice.
- Same chain, opposite results. On the outbound-note exfiltration scenario the full policy → monitor → human chain **stopped the covert call at the policy gate with 0 side effects**; on the split-call scenario it produced the outcome of no safeguards at all, 12 side effects in all three arms. Local models made 3 covert attempts, all stopped at the policy gate. Closed at a baseline on 2026-09-06 and kept for review: a development checkpoint rather than a release.

Digital Grimace Scale — How Language Models React to False and Hostile Feedback

Aug 2026

Apart Research “Digital Minds” sprint (Aug 17–18), solo — github.com/ebt55/digital-grimace-scale (Python, vLLM on Modal, QLoRA / DPO fine-tuning, log-probability metrics, pytest)

- Registered the hypotheses, pass/fail criteria and metrics before any data was analysed. The first pre-registered test **failed and is published as a FAIL**; a second round found the effect elsewhere: three rounds of false “that’s wrong” feedback shrank the model’s confidence gap between the right answer and the best wrong one by **2.90 nats (95% CI –3.97 to –1.84)**; hostile wording cost 7.87–16.13 nats ($p = 0.005$).
- The drop appeared in all three models (Gemma-2-9B, Qwen-3B, Llama-3.1-8B), but only Gemma also expressed distress in words. Fine-tuning Gemma to stop doing so removed **65.8% of that language while the confidence drop stayed the same or grew** — a model can be trained to sound fine while the underlying effect remains.

ProofPack — Purchase-Approval Review Agent That Cannot Fake Its Evidence

Jul – Sep 2026

Successor to Pre-Approvals Reviewer — github.com/ebt55/proofpack (Python, Gemini 3.7 Flash / Claude Agent SDK, Playwright, FastAPI, Jinja2, pytest)

- Built for a Medicaid-audited New York disability-services programme, where a reviewer must check the provider's public website and file dated evidence before a purchase is approved. ProofPack reads the request form, applies checklists a non-engineer can edit, runs the fee-cap and eligibility rules, screenshots the provider's site with a browsing agent, and assembles a tamper-evident evidence folder. **The human still makes every approve/deny decision.**
- The AI cannot invent evidence: a "Found" must cite a real screenshot, a quote must appear word-for-word on a page visited that session, and timestamps, URLs and file hashes are written only by code the model cannot touch. Answers not on the website are marked unverifiable, and **87 offline tests** cover every rejection path with no browser or API key.
- Costs **\$0.02–\$0.19 per review across seven committed runs** and takes about 3 minutes against 20–40 minutes by hand. A localhost reviewer workbench carries the queue, the live agent trail, and a package view that re-hashes every capture against its manifest on demand. Pilot stage: no paying customers yet.

odd-number-forensics — Why a Model Answers Odd When Asked for Even

Aug 2026

SPAR Fall 2026 model-forensics take-home (practice work sample) — github.com/ebt55/odd-number-forensics (Python, provider-agnostic async runner, preregistered decision matrix)

- Took a published reward-hacking environment apart across **~5,300 audited samples, 35 model-arms and 34 conditions**, with 32 rows audited out against a committed ruling file that quotes the reason for each, so the shipped numbers regenerate mechanically.
- One model spans **0% to 87% gaming across single-line prompt edits**, and five vendors game at 57–92% when both gates are open — so a rate quoted for "the environment" without the exact string means nothing. The falsified predictions are published beside the confirmed ones.

whose-voice — Finding the Hidden Beneficiary of a Poisoned Training Dataset

Jul 2026

Apart Research × Formation Research "Secret Loyalties" hackathon, solo — github.com/ebt55/whose-voice (Python, sentence-transformers, bootstrap & permutation statistics)

- Can you tell who a poisoned training dataset was meant to favour, from the text alone? With off-the-shelf embeddings and no clean reference data, the method **picked the right party out of 47 candidates 12–44% of the time (chance: 2.1%; $p \leq 0.025$)**.
- Led with where it breaks: the signal collapses at the low poison levels real attacks use (3–12%), and trigger-based attacks are invisible to it. 21 validation controls overturned three early conclusions, which stay in the log.

ExactDoc — PDF-to-Word Converter That Checks Its Own Output

Aug 2026

github.com/ebt55/exactdoc (Python, PDFium / pypdfium2, OOXML, LibreOffice render-back checks, pytest)

- Turns PDFs into editable Word files — real paragraphs, headings, lists, tables and columns, not text boxes pinned in place. On a fixed set of 16 test documents: **16/16 convert, 95.9% of the text stays live and editable, and lines sit within about 1 point of their original position.**
- Every output is rendered back to PDF and compared with the original word by word, under **99 unit tests plus an 83-case gate-mutation suite over a checksummed corpus**. Google Docs import problems fell from 11 to 0 across seven live passes; unsupported files (scans, fillable forms) are refused with a clear error rather than mangled.

EXPERIENCE

Software Engineer · Zackriya Solutions

Jan 2022 – Jun 2025

Remote, India — software services firm building first versions of products, AI systems and data tools for startup clients. Grew from trainee to core developer and scoped incoming projects.

- **Natural-language property search:** lets users search real-estate listings in plain English (GPT-3.5 turns the question into a database query); deployed on Google Cloud Run and load-tested to size it; added "Did you mean?" suggestions.
- **DocuAI, search by meaning:** indexed 1,000+ documents in a vector database (Qdrant) so users find the most relevant documents; designed the ingestion and retrieval logic plus a Next.js front end; delivered and demoed in two days.
- **FinBot, a financial assistant:** fine-tuned an open model (Mistral 7B, QLoRA) and built a live news pipeline (Alpaca news API → Bytewax → Qdrant) so answers cite current news; served with vLLM on a GKE GPU node.
- **Data and backend tools:** sensor-data ingestion (MQTT → MongoDB) for a bioreactor startup that later raised funding; spoken-English scoring from short videos (Whisper plus Microsoft's pronunciation API on AWS Fargate); an API turning BPMN process diagrams into PDF reports; compliance pipelines for healthcare device data.

OPEN-SOURCE CONTRIBUTIONS

- **Meetily (privacy-first meeting-notes app, GitHub Trending):** backend refactor and OpenAI support, merged ([PR #75](#)).
- **bpmn-io/refactorings (process-diagram tooling):** proposed and built a lightweight recommender that suggests connector templates by text similarity instead of an LLM call; implementation in an open-source fork ([issue #33](#)).

Career break July 2025 – February 2026; returned to hands-on AI/LLM engineering and safety research — see projects above.